

1 State Representation

1.1 Card Encoding

Each card is represented as a two-character string (rank, suit), e.g. "Ah". Internally, a `Card` object stores:

- `rank_char` $\in \{2, \dots, 9, T, J, Q, K, A\}$,
- `suit` $\in \{h, d, c, s\}$,
- `rank` $\in \{2, \dots, 14\}$, obtained via the map

$$\text{rank}(r) = \text{index}_{\text{"23456789TJQKA"}}(r) + 2.$$

This integer encoding is the standard trick that lets straights, kickers, and comparisons reduce to ordinary integer arithmetic instead of string parsing. Dunder methods (`--eq--`, `--lt--`, `--gt--`, `--hash--`) are implemented on rank (and, for equality/hashing, suit) so that Python’s built-in `sorted()`, set membership, and comparison operators work directly on `Card` objects.

1.2 Hand-Strength Feature: Monte Carlo Equity

Rather than hand-crafting dozens of categorical features (pair/no-pair, suitedness, connectivity, etc.), the state is collapsed into a single continuous scalar:

$$\hat{p}_{\text{win}} = \text{estimate_equity}(\text{hole}, \text{board}, N) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[H_{\text{me}} > H_{\text{opp}}^{(i)}] + \frac{1}{2} \mathbb{1}[H_{\text{me}} = H_{\text{opp}}^{(i)}],$$

where, for each of N Monte Carlo rollouts, an opponent hole-card pair and the remaining community cards are sampled without replacement from the deck minus all known cards, and $H_{(\cdot)}$ is the best 5-of-7 hand rank returned by the evaluator (Section 2). This is precisely the definition of hand equity used in the poker literature, and it is the sufficient statistic the rest of the decision policy conditions on. Collapsing state to one number is a deliberate simplification: it sacrifices the ability to reason about *equity distribution shape* (e.g. a hand that is a robust 55% vs. a polarized hand that is 90% or 10%) in exchange for a small, tractable, hand-independent policy.

2 Hand Evaluation Algorithm

Given 7 cards, the agent must find the best 5-card poker hand. This is implemented in two layers:

1. `eval_5`: scores an exact 5-card hand as a tuple

$$(\text{category}, \text{tiebreak}_1, \text{tiebreak}_2, \dots),$$

where `category` $\in \{0, \dots, 8\}$ (High Card $\rightarrow$ Straight Flush) and the remaining entries are the ranks needed to break ties within a category (e.g. trip rank then kickers for Three of a Kind). Because Python tuples compare lexicographically, hand comparison reduces to `tuple_a > tuple_b`, which automatically and correctly implements both category ranking and kicker resolution.

Straights are detected by scanning the five *unique* ranks in descending order for a run of five consecutive integers, with a special case for the “wheel” (A-2-3-4-5, which ranks as a 5-high straight). *Flushes* are detected via set cardinality of suits. Pairs/trips/quads are counted with `collections.Counter`, sorted by (count, rank) so the most significant grouping is compared first.

2. `best_hand_7`: enumerates all $\binom{7}{5} = 21$ five-card sub-combinations via `itertools.combinations` and takes the maximum `eval_5` score.

2.1 Correctness Validation

The evaluator was unit-tested against one representative hand from every category (royal/straight flush, quads, full house, flush, straight including the wheel, trips, two pair, one pair, high card) and against a tie scenario (two identical pocket pairs on a shared board, verifying kicker resolution). All categories were classified and ordered correctly; see Table 1.

Table 1: Hand-evaluator unit tests (input $\rightarrow$ returned category tuple)

Input cards	Expected category	Returned score
Ah Kh Qh Jh Th	Straight Flush (Royal)	(8, 14)
2c 2d 2h 2s 5c	Four of a Kind	(7, 2, 5)
3c 3d 3h 5s 5c	Full House	(6, 3, 5)
2h 5h 9h Jh Kh	Flush	(5, 13, 11, 9, 5, 2)
4c 5d 6h 7s 8c	Straight	(4, 8)
Ah 2d 3h 4s 5c	Straight (wheel)	(4, 5)
9c 9d 9h 2s 5c	Three of a Kind	(3, 9, 5, 2)
9c 9d 2h 2s 5c	Two Pair	(2, 9, 2, 5)
9c 9d 2h 5s 7c	One Pair	(1, 9, 7, 5, 2)
9c 5d 2h Ks 7c	High Card	(0, 13, 9, 7, 5, 2)

3 Decision Policy

The policy is a threshold rule on $\hat{p}_{\text{win}}$, applied differently depending on whether the agent is facing a bet.

3.1 Facing No Bet (`amount_to_call` = 0)

$$\text{action} = \begin{cases} \text{RAISE} & \hat{p}_{\text{win}} > \theta_{\text{value}} = 0.60 \\ \text{RAISE (w.p. 0.40)} & \theta_{\text{thin}} = 0.50 < \hat{p}_{\text{win}} \leq \theta_{\text{value}} \\ \text{CALL (check)} & \text{otherwise.} \end{cases}$$

The stochastic “thin value” raise is a deliberate variance/unpredictability injection: an opponent who observes deterministic behavior at a fixed equity threshold can exploit it by folding correctly whenever the agent bets. Randomizing the raise frequency in the 50–60% equity band makes the agent’s action distribution partially decorrelated from its true hand strength, i.e. an elementary, hand-crafted analogue of a mixed (game-theoretically balanced) strategy.

3.2 Facing a Bet (`amount_to_call` > 0)

The agent first computes the pot odds required to profitably call:

$$\pi_{\text{req}} = \frac{\text{amount_to_call}}{\text{pot_size} + \text{amount_to_call}},$$

which is the break-even win probability derived directly from the EV of calling,

$$\text{EV}(\text{call}) = \hat{p}_{\text{win}} \cdot (\text{pot_size} + \text{amount_to_call}) - \text{amount_to_call} \geq 0 \iff \hat{p}_{\text{win}} \geq \pi_{\text{req}}.$$

The policy is then:

$$\text{action} = \begin{cases} \text{RAISE (or CALL if capped)} & \hat{p}_{\text{win}} > \theta_{\text{value}} = 0.60 \\ \text{RAISE (w.p. 0.30) else CALL} & \hat{p}_{\text{win}} > \pi_{\text{req}} + 0.03 \\ \text{RAISE (w.p. 0.07)} & \theta_{\text{bluff}} = 0.35 < \hat{p}_{\text{win}} \leq \pi_{\text{req}} + 0.03 \\ \text{FOLD} & \text{otherwise.} \end{cases}$$

The +0.03 margin above π_{req} is a conservative buffer that absorbs Monte Carlo sampling noise in $\hat{p}_{\text{win}}$ (see Section 4), so the agent does not flip between CALL and FOLD purely due to simulation variance near the indifference point. The low-frequency (7%) raise below the profitable-call threshold is a bounded bluffing element: it prevents the agent from being perfectly “face up” (never raising without genuine equity), at a bounded EV cost, since it fires rarely and only when $\hat{p}_{\text{win}}$ still exceeds a floor of 0.35.

3.3 Pre-Flop Fast Path

Because pre-flop equity from only 80 rollouts is noisy (small sample, and pre-flop hand strength is well understood analytically), two rule-based shortcuts bypass the Monte Carlo estimate entirely:

- Pocket pairs of rank 99 or higher instantly RAISE (these hands are > 80% favorites against a random hand pre-flop and rarely benefit from the added variance of an equity estimate).
- Two cards both rank $\geq T$ (e.g. Broadway cards) CALL cheap bets (≤ 4 chips).

All other pre-flop holdings fall through to the general Monte Carlo policy above.

4 Complexity Analysis

4.1 Per-Rollout Cost

Each Monte Carlo rollout calls `best_hand_7` twice (once for the agent, once for the sampled opponent), and each call enumerates $\binom{7}{5} = 21$ combinations, each scored in $O(1)$ (bounded work independent of input size, since a poker hand is always exactly 5 cards). So one rollout costs $O(1)$, and N rollouts cost $O(N)$.

4.2 Simulation Budget by Street

$$N(\text{street}) = \begin{cases} 80 & \text{pre-flop (fallback only; fast path above dominates)} \\ 250 & \text{flop} \\ 300 & \text{turn} \\ 400 & \text{river} \end{cases}$$

The budget increases as the street progresses because (a) fewer unknown cards remain, keeping wall-clock time roughly comparable across streets despite the larger N , and (b) later streets carry higher pot commitment, so a more precise equity estimate is worth the extra compute.

4.3 Empirical Timing

On the evaluation container (single core, CPython, no vectorization), a full `estimate_equity` call measured:

- Turn (300–400 rollouts): ≈ 77 ms/call.
- River (400 rollouts): ≈ 78 ms/call.

A single hand can invoke `get_action` up to roughly 6–8 times if both players raise on every street up to the betting cap. Over a 10,000-hand round-robin match this implies a worst-case compute budget on the order of $10^4 \times 8 \times 0.08 \text{ s} \approx 1.8$ hours *per opponent*, which is a real risk in a multi-agent round-robin bracket with a wall-clock or per-match timeout. This is the single most important engineering risk in the current implementation, ahead of any strategic weakness.

5 Reward / Regret Shaping

This agent does not use reinforcement learning, so there is no learned reward signal to shape. Instead, the analogous design choice is the *decision threshold vector* $(\theta_{\text{value}}, \theta_{\text{thin}}, \theta_{\text{bluff}}, \text{margin})$, which plays the same role that a reward function plays in an RL formulation: it defines what the agent treats as “good enough” evidence to commit more chips. The thresholds are grounded directly in the EV inequality of Section 3.2 rather than fit to data, which makes the policy interpretable and immune to reward mis-specification, at the cost of not being adaptive to a specific opponent’s tendencies.